

PORTADA

Arquitectura de Software

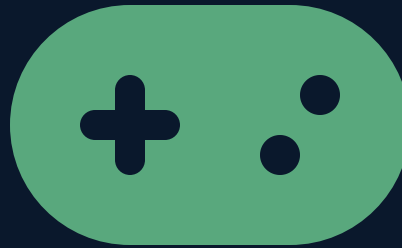
Semana 4 — Requisitos No Funcionales: Norma ISO 25010

Universidad de Antioquía · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

**50 veces más tráfico
del que se esperaba**



Pokémon GO, julio de 2016

Producto	Pokémon GO — juego móvil de realidad aumentada, Niantic
Lanzamiento	6 de julio de 2016 (Estados Unidos, Australia, Nueva Zelanda)
Demanda observada	Muy por encima de las proyecciones de capacidad del equipo — Niantic reportó públicamente picos de tráfico varias veces mayores a lo estimado
Síntoma	Servidores caídos, tiempos de carga excesivos, errores de autenticación e imposibilidad de jugar durante horas
Respuesta	Escalado de emergencia en infraestructura, con apoyo directo de Google Cloud (Niantic había sido parte de Google)
Resultado funcional	El juego —mecánicas, mapas, captura de criaturas— funcionaba correctamente cuando el usuario lograba entrar

Caso ampliamente cubierto por prensa tecnológica contemporánea (2016) y declaraciones públicas de Niantic.

CASO · LO QUE NO FALLÓ

La funcionalidad estaba completa

Esto es lo que distingue este caso de HealthCare.gov (Semana 1): **Pokémon GO no tenía un problema de funcionalidad incorrecta.** Las reglas del juego, la captura de criaturas, el mapa con geolocalización — todo **hacía lo que debía hacer.**

El sistema fallaba en algo que no tiene que ver con "qué hace" el software, sino con "cómo se comporta" bajo condiciones reales de uso.

CASO · LO QUE SÍ FALLÓ

La capacidad de soportar la demanda real

El equipo de Niantic diseñó su infraestructura para un volumen de usuarios estimado a partir de lanzamientos previos de la empresa (Ingress, un juego similar con audiencia mucho menor). **La demanda real de Pokémon GO desbordó ese cálculo por un margen enorme.**

- Los servidores no podían atender la cantidad de solicitudes simultáneas.
- El sistema de autenticación se volvió, por sí solo, un cuello de botella.
- No había un plan de escalado automático preparado para ese nivel de crecimiento.

TRANSICIÓN

Un tipo de requisito que no es "qué hace"

Pokémon GO cumplía perfectamente sus **requisitos funcionales** — lo que el sistema debe hacer. Lo que faltó fue atender con suficiente rigor un **requisito no funcional**: qué tan bien debía comportarse el sistema bajo una carga de usuarios varias veces mayor a la esperada.

Hoy formalizamos esa distinción y la norma que la estructura: ISO/IEC 25010.

CONCEPTO

Requisito funcional vs. no funcional

	Requisito funcional	Requisito no funcional
Responde	¿Qué debe hacer el sistema?	¿Qué tan bien debe hacerlo?
Ejemplo	"El usuario puede capturar un Pokémon al tocar la pantalla"	"El sistema debe responder en menos de 2 segundos con 1 millón de usuarios simultáneos"
Se valida con	Pruebas funcionales, casos de uso	Pruebas de carga, rendimiento, seguridad

Pokémon GO cumplía la columna izquierda casi perfectamente. Falló en la columna derecha.

CONCEPTO

Por qué los NFR son decisiones arquitectónicas

Un requisito no funcional casi nunca se resuelve en un solo componente: **afecta la estructura completa del sistema**, tal como se definió "decisión arquitectónica" en la Semana 3.

- Soportar más usuarios exige decisiones de escalado que atraviesan todo el sistema, no un ajuste local.
- La seguridad no se "agrega" a un módulo — depende de cómo están diseñadas las fronteras entre todos los componentes.
- Corregir un NFR después del diseño inicial —como tuvo que hacer Niantic, bajo presión— es mucho más costoso que haberlo considerado desde el principio.

CONCEPTO · LA NORMA

ISO/IEC 25010

ISO/IEC 25010 es el estándar internacional que organiza los requisitos no funcionales en un **modelo de calidad de producto**, con 8 características medibles, para que dejen de ser criterios vagos ("que sea rápido", "que sea seguro") y se conviertan en algo que se puede especificar y verificar.

Esta norma también se estudia en el curso de Calidad de Software desde la perspectiva de pruebas; aquí la abordamos desde la perspectiva de **decisión arquitectónica**.

CONCEPTO

Las 8 características, resumidas

Característica	Pregunta que responde
Adecuación funcional	¿Hace lo que el usuario necesita, de forma completa y correcta?
Eficiencia de desempeño	¿Usa bien el tiempo y los recursos, incluso bajo carga alta?
Compatibilidad	¿Coexiste e intercambia información con otros sistemas?
Usabilidad	¿Es fácil de entender, aprender y operar?
Confiabilidad	¿Se mantiene funcionando ante fallos y bajo condiciones normales?
Seguridad	¿Protege datos e información de accesos no autorizados?
Mantenibilidad	¿Es fácil de modificar sin introducir defectos?
Portabilidad	¿Se puede transferir a otro entorno de hardware o software?

CONCEPTO · MIRADA ARQUITECTÓNICA

Cuáles dependen más de la arquitectura

No las 8 características pesan igual en el diseño estructural del sistema. Algunas se resuelven principalmente con **decisiones arquitectónicas**; otras dependen más de la calidad del código y la interfaz.

Con fuerte peso arquitectónico	Con más peso en código/UI
Eficiencia de desempeño, Confiabilidad, Seguridad, Compatibilidad, Portabilidad	Usabilidad, parte de Mantenibilidad

Pokémon GO falló exactamente en la columna izquierda: eficiencia de desempeño y confiabilidad bajo carga.

CONTRAEJEMPLO

Mito: "los NFR se agregan al final"

Es común tratar los requisitos no funcionales como un ajuste de última hora: "primero que funcione, después lo optimizamos".

El caso Pokémon GO muestra el límite de esa estrategia: para cuando el problema de escalabilidad se hizo evidente —el día del lanzamiento, con millones de usuarios reales— ya era demasiado tarde para rediseñar la arquitectura sin causar una crisis pública.

Los NFR con fuerte peso arquitectónico deben decidirse al mismo tiempo que la estructura del sistema, no después.

CONCEPTO

Cómo se especifica un NFR medible

Un requisito no funcional bien escrito no dice "que sea rápido" — dice algo **verificable**:

"El sistema debe responder al 95 % de las solicitudes en menos de 2 segundos, con hasta 10.000 usuarios concurrentes."

Ese nivel de precisión es lo que le faltó al plan de capacidad original de Niantic: sin un número concreto de usuarios esperados, no hay forma de diseñar (ni de probar) una arquitectura que lo soporte.



TENSIÓN CONCEPTUAL

¿Se puede diseñar sin conocer los NFR desde el inicio?

En proyectos reales —incluido el proyecto de Fábrica Escuela— es frecuente empezar a construir con requisitos funcionales claros, pero con los no funcionales apenas esbozados ("que sea rápido", "que sea seguro", sin números).

¿Es razonable avanzar con la arquitectura antes de tener NFR medibles, confiando en que se pueden ajustar después?

¿O eso es exactamente el error que cometió Niantic?



INTERACCIÓN

Funcional o no funcional

Clasifiquen cada requisito y, si es no funcional, digan a qué característica de ISO 25010 corresponde:

1. "El sistema debe permitir registrar un nuevo estudiante con nombre, correo y programa."
2. "El sistema debe soportar 5.000 usuarios simultáneos sin degradar el tiempo de respuesta."
3. "Solo el personal autorizado puede modificar las calificaciones de un estudiante."
4. "El código debe poder migrarse de un proveedor de nube a otro sin reescribir la lógica de negocio."

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. Un **requisito funcional** dice qué hace el sistema; un **requisito no funcional** dice qué tan bien lo hace.
2. Pokémon GO (2016) demuestra que un sistema funcionalmente perfecto puede fallar por completo si un NFR —en este caso, capacidad bajo carga— no se cumple.
3. **ISO/IEC 25010** organiza los NFR en 8 características medibles del modelo de calidad de producto.
4. No todas las características pesan igual en la arquitectura: eficiencia, confiabilidad, seguridad, compatibilidad y portabilidad dependen fuertemente de decisiones estructurales.
5. Un NFR útil se especifica con **números verificables**, no con adjetivos vagos como "rápido" o "seguro".

INSTRUCCIONES

Taller de hoy — Implementación de ejemplo

Demostración guiada por el docente — 30 a 40 minutos

El docente muestra en vivo, sobre el mismo backend Spring Boot de sesiones anteriores, cómo una decisión de **caché** — motivada por un requisito no funcional de eficiencia de desempeño— cambia la arquitectura del componente sin cambiar su funcionalidad.

Un NFR que cambia la arquitectura, sin cambiar la función

```
// SIN considerar el NFR de eficiencia de desempeño:  
// cada solicitud repite el mismo trabajo costoso.  
public class SaludoService {  
    public String generar() {  
        return repo.buscarMensaje(); // consulta cada vez  
    }  
}  
  
// CON el NFR "responder en menos de 200 ms incluso  
// bajo carga alta" incorporado desde el diseño:  
@Service  
public class SaludoService {  
    @Cacheable("saludos") // decisión arquitectónica: caché  
    public String generar() {  
        return repo.buscarMensaje(); // solo se ejecuta una vez  
    }  
}
```

El resultado funcional es idéntico — el mensaje que ve el usuario no cambia. Lo que cambió es la arquitectura interna, exigida por un requisito no funcional explícito.

No hay entregable de estudiantes: el objetivo es reconocer cuándo un NFR obliga a una decisión estructural.

CIERRE

Para la próxima semana — y cierre de Unidad 1

- Definir, para el proyecto propio de Fábrica Escuela, al menos un requisito no funcional medible (no un adjetivo vago).
- Repasar los conceptos de conector y protocolo — se formalizan la próxima semana.

La Semana 5 cierra la Unidad 1: conectores y protocolos de Garlan y Shaw, y el stack tecnológico (MEAN, MERN, LAMP, WAMP, XAMP) — el punto donde toda la teoría de estas cinco semanas se traduce en decisiones concretas de tecnología para el Sprint 1.

